

Spark Declarative Pipelines

Hands-On Lab — Setup Guide

Overview

This lab walks you through building a four-layer medallion pipeline using Spark Declarative Pipelines (SDP) SQL. You will create two silver tables and two gold tables, connected in a DAG that SDP infers automatically from your SQL.

	Description
silver_customers	Typed, deduplicated customer dimension from bronze CSV data
silver_orders	Incrementally-ingested, validated orders fact table
gold_customer_summary	Customer 360 rollup: lifetime revenue, order counts, recency
gold_region_summary	Region-level KPIs stacked on top of the customer summary

Before you start

Run the bootstrap notebook (00_bootstrap_bronze.py) first. It creates the two bronze source tables you need. Set the catalog and schema widgets to your personal sandbox before running it — the same values you will enter in Step 3 below.

Part 1 — Create the Pipeline

Follow these steps to create and configure a new ETL pipeline in the Databricks UI.

#	Action	Details
1	Open the New menu	In the left sidebar, click + New. From the menu that appears, select ETL pipeline. This opens a blank Lakeflow Pipelines editor with a default .sql file already created.

2	Name your pipeline	At the top of the editor, click the default pipeline name (e.g. "New Pipeline 2026-05-18") to edit it. Give it a descriptive name such as your initials or team prefix followed by <code>_sdp_lab</code> — for example: <pre>jsmith_sdp_lab</pre> Press Enter or click away to confirm.
3	Set the default catalog and schema	Click Settings in the pipeline toolbar (top of the editor). Under Destination, use the catalog and schema dropdowns to select your personal sandbox: Catalog: <code><your team catalog></code> Schema: <code><your sandbox schema></code> This tells SDP where to write the output tables. Your silver and gold tables will be created here automatically when the pipeline runs. Click Save.
4	Add silver_customers.sql	Click the + tab next to the existing <code>.sql</code> file in the editor to add a new file, or rename the default file. Name it <code>silver_customers.sql</code>. Paste the <code>silver_customers</code> query from the Query Reference section of this handout. Update the FROM clause to use your fully-qualified bronze table: <pre><catalog>.<schema>.dim_customers_raw.</pre>
5	Dry run — check the graph	Click Dry run in the pipeline toolbar. SDP parses your SQL and renders the pipeline DAG — you should see <code>silver_customers</code> appear as a single node. Dry run validates syntax and resolves table references without writing any data. Use it after each file you add. Fix any red nodes before moving on.
6	Add silver_orders.sql	Add a second <code>.sql</code> file named <code>silver_orders.sql</code>. Paste the <code>silver_orders</code> query and update its FROM clause: <pre>FROM STREAM(<catalog>.<schema>.raw_customer_orders)</pre> Dry run again — you should now see two silver nodes.
7	Add gold_customer_summary.sql	Add <code>gold_customer_summary.sql</code> and paste the query. No source path changes needed here — this query references <code>silver_customers</code> and <code>silver_orders</code> by short name, which SDP resolves automatically within the pipeline. Dry run — the DAG should now show <code>silver_customers</code> and <code>silver_orders</code> feeding into <code>gold_customer_summary</code>.

8	Add gold_region_summary.sql	Add gold_region_summary.sql and paste the final query. Again, no path changes — it references gold_customer_summary by short name. Dry run one more time. You should now see the full four-node DAG: <pre>bronze → silver_customers → gold_customer_summary → gold_region_summary</pre> <pre>silver_orders ↗</pre>
9	Run the pipeline	Click Run pipeline (top-right, blue button) to execute a full pipeline run. SDP will run nodes in DAG order: both silver tables first (they can run in parallel), then gold_customer_summary, then gold_region_summary. When complete, check the event log in the pipeline UI for constraint violation counts — compare against the expected DQ table in the bootstrap notebook.

Part 2 — Explore the Results

Once the pipeline run completes, open Catalog Explorer and navigate to your sandbox schema. You should see four new tables. Try the following queries in a notebook or SQL editor to verify the output:

```
-- How many rows survived each silver layer?

SELECT COUNT(*) FROM silver_customers;  -- expect 10 (of 13 input rows)

SELECT COUNT(*) FROM silver_orders;      -- expect 26 (of 31 input rows)


-- Confirm the gold rollup looks right

SELECT * FROM gold_customer_summary ORDER BY lifetime_revenue DESC LIMIT 5;


-- Region-level check

SELECT * FROM gold_region_summary ORDER BY total_revenue DESC;
```

Check the pipeline event log

In the pipeline UI, click on any node after a run to see its constraint violation counts. `silver_customers` should show 1 drop (`valid_customer_id`) and 1 warning (`valid_payment_terms` — the row is kept because the constraint has no `ON VIOLATION` clause). The row count goes $13 \rightarrow 10$: 1 constraint drop, plus 1 duplicate removed by the `ROW_NUMBER()` CTE, plus 1 inactive customer filtered by `WHERE is_active = TRUE`. `silver_orders` should show 5 drops (2 NULL `order_id` + 1 NULL `customer_id` + 2 bad `qty`), landing at 26 of 31 input rows. These numbers are your proof that the data quality layer is working.

Spark Declarative Pipelines

SQL Syntax Quick Reference

1. Table Types

SDP gives you two first-class table types. The right choice depends on whether your source data is append-only or whether you need to handle updates and aggregations.

	When to use it
<pre>CREATE OR REFRESH MATERIALIZED VIEW name AS ...</pre>	Uses batch semantics. SDP attempts incremental refresh automatically — only processing upstream changes since the last run. Falls back to a full recompute only when the query can't be incrementally updated. Best choice for complex transformations, enrichment joins, and aggregations (silver joins, gold rollups). Example: silver_customers, gold_customer_summary
<pre>CREATE OR REFRESH STREAMING TABLE name AS SELECT * FROM STREAM(source_table)</pre>	Uses streaming semantics. Each input row is processed exactly once — the engine never re-reads rows it has already consumed. Best choice for ingestion and row-level transformations on append-only sources (orders, events, logs). STREAM() is the SQL function that marks the source as a stream. Example: silver_orders

2. Data Quality Constraints (EXPECT)

Constraints are declared inside the CREATE statement, before the AS SELECT. Each one names a condition and an action.

	Meaning
<pre>CONSTRAINT <name></pre>	Rows that fail the condition are silently removed from the output.
<pre>EXPECT (<condition>)</pre>	Use for missing primary keys, impossible values — anything you can't reason about downstream.
<pre>ON VIOLATION DROP ROW</pre>	

CONSTRAINT <name>	Any violation stops the pipeline update and atomically rolls back the transaction.
EXPECT (<condition>)	Use for hard business rules where bad data must never reach downstream.
ON VIOLATION FAIL UPDATE	
CONSTRAINT <name>	Rows are kept and the violation is counted in the pipeline event log (this is the warn policy — the default).
EXPECT (<condition>)	Use for 'data smell' checks — you want visibility without blocking delivery.

Tip — Constraint names matter

Pick descriptive names (valid_customer_id, plausible_price). They appear in the pipeline UI and event logs, making it easy to spot which rule is firing without digging into SQL.

3. Intra-Pipeline Table References

Tables defined in the same pipeline can be referenced by short name — no catalog.schema prefix needed. SDP resolves them automatically and uses those references to build the DAG (execution order).

```
-- gold layer can reference silver by short name:

FROM silver_customers AS c

LEFT JOIN silver_orders AS o ON c.customer_id = o.customer_id

-- SDP infers: silver must run before gold.

-- You do NOT need to manually order your pipeline steps.
```

4. Safe Type Casting — TRY_CAST

Bronze tables loaded from CSV land with every column as STRING. TRY_CAST converts to the target type and returns NULL (instead of erroring) on bad values. Pair with a constraint to handle the NULLs.

```
TRY_CAST(payment_terms_days AS INT)          -- '30' -> 30, 'abc' -> NULL
TRY_CAST(order_date AS DATE)                  -- '2024-01-15' -> date, 'bad' -> NULL
TRY_CAST(unit_price AS DECIMAL(10,4)) / 1000  -- cast then scale-correct

-- Booleans from string flags:
LOWER(is_active) IN ('true','t','1','yes')    -- IN already returns BOOLEAN
```

5. Row Deduplication with ROW_NUMBER()

Bronze layers often contain duplicates from re-ingested files or CDC. A ROW_NUMBER() window function inside a CTE is the standard pattern to keep exactly one row per key.

```
WITH deduped AS (
  SELECT *
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY customer_name) AS rn
    FROM typed
  )
  WHERE rn = 1  -- keep the 'first' row per customer_id
)
SELECT ... FROM deduped
```

Note — Streaming tables and window functions

Window functions like ROW_NUMBER() require batch semantics — they can't run directly on a STREAM() source because streaming doesn't support the unbounded ordering a window function needs. This is one reason silver_customers is a MATERIALIZED VIEW rather than a STREAMING TABLE.

6. GROUP BY ALL

GROUP BY ALL is a Databricks SQL shorthand that automatically groups by every non-aggregated column in the SELECT list. It keeps gold-layer aggregation queries concise.

```
SELECT

  c.customer_id, c.customer_name, c.tier, c.region, c.contract_type,

  COUNT(DISTINCT o.order_id) AS order_count,

  SUM(o.gross_revenue)      AS lifetime_revenue

FROM silver_customers c

LEFT JOIN silver_orders o ON c.customer_id = o.customer_id

GROUP BY ALL;  -- equivalent to GROUP BY customer_id, customer_name, tier, region,
contract_type
```

7. Division-by-Zero Guard — NULLIF

When computing ratios in gold, protect against divide-by-zero with NULLIF(denominator, 0). It returns NULL instead of an error when the denominator is zero.

```
-- Revenue per active customer in a region:

ROUND(SUM(lifetime_revenue) / NULLIF(COUNT(*), 0), 2) AS revenue_per_customer

--
--
--      returns NULL if COUNT(*) = 0, avoiding ZeroDivisionError
```

8. Table & Column Comments

SDP surfaces table-level COMMENT strings in the Unity Catalog UI, making your pipeline self-documenting.

```
CREATE OR REFRESH MATERIALIZED VIEW silver_customers

COMMENT "Cleaned, typed, deduplicated customer master. One row per active customer."

AS SELECT ...
```

Full documentation: docs.databricks.com/en/dlt/index.html

Pipeline Query Reference

Full annotated SQL for all four pipeline layers

Pipeline Overview

Your pipeline has four nodes. SDP infers execution order from table references — you don't configure dependencies manually.

	Type
	Source
silver_customers	MATERIALIZED VIEW <catalog>.<schema>.dim_customers_raw
silver_orders	STREAMING TABLE STREAM(<catalog>.<schema>.raw_customer_orders)
gold_customer_summary	MATERIALIZED VIEW silver_customers + silver_orders
gold_region_summary	MATERIALIZED VIEW gold_customer_summary

silver_customers

Type: MATERIALIZED VIEW | **Source:** <catalog>.<schema>.dim_customers_raw

Casts every column from STRING to its real type, deduplicates on customer_id, and filters to active customers only. A MATERIALIZED VIEW is used here because the transformation involves deduplication logic (a window function) that isn't compatible with streaming semantics — and because SDP will handle incremental refresh automatically whenever the source changes.

What to notice

Two constraints with different actions: `valid_customer_id` drops bad rows immediately; `valid_payment_terms` only counts violations (no `ON VIOLATION` clause). The `ROW_NUMBER()` CTE handles deduplication before the constraint can even fire.

```
CREATE OR REFRESH MATERIALIZED VIEW silver_customers
(
  -- ON VIOLATION DROP ROW    -> silently remove the bad row
  -- ON VIOLATION FAIL UPDATE -> stop the pipeline, roll back the transaction
  -- (no clause / warn)      -> count the violation, keep the row
  CONSTRAINT valid_customer_id  EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_payment_terms EXPECT (payment_terms_days BETWEEN 0 AND 365)
)
COMMENT "Cleaned, typed, deduplicated customer master. One row per active customer."
AS
WITH typed AS (
  SELECT
    customer_id, customer_name, tier, region, contract_type,
    TRY_CAST(payment_terms_days AS INT) AS payment_terms_days,
    LOWER(is_active) IN ('true','t','1','yes') AS is_active
  FROM <catalog>.<schema>.dim_customers_raw
),
deduped AS (
  SELECT *
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY customer_name) AS rn
    FROM typed
  )
  WHERE rn = 1
)
SELECT customer_id, customer_name, tier, region, contract_type,
       payment_terms_days, is_active
FROM deduped
WHERE is_active = TRUE;
```

silver_orders

Type: STREAMING TABLE | **Source:** STREAM(<catalog>.<schema>.raw_customer_orders)

Casts and validates the orders fact table, applies a scale correction to unit_price, and computes gross_revenue and total_freight_cost as derived columns. A STREAMING TABLE is used because the orders source is append-only — incremental processing means each run only reads new rows.

What to notice

STREAM() wraps the source table name. The unit_price /1000 correction is documented explicitly and applied once here in silver so gold layers never need to know about it. The plausible_price constraint has no ON VIOLATION clause — it observes without blocking.

```

CREATE OR REFRESH STREAMING TABLE silver_orders
(
  CONSTRAINT has_order_id      EXPECT (order_id      IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT has_customer_id   EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT has_positive_qty  EXPECT (quantity_units > 0)      ON VIOLATION DROP ROW,
  -- No action (warn): row kept, violation counted in event log.
  CONSTRAINT plausible_price   EXPECT (unit_price BETWEEN 0 AND 1)
)

COMMENT "Typed, validated orders with revenue + freight. unit_price scale-corrected /1000 from
bronze."

AS

SELECT
  order_id,
  TRY_CAST(order_date AS DATE)                AS order_date,
  customer_id, product_id, plant_id,
  TRY_CAST(quantity_units AS INT)              AS quantity_units,
  TRY_CAST(unit_price AS DECIMAL(10,4)) / 1000 AS unit_price,
  TRY_CAST(freight_cost_per_unit AS DECIMAL(10,4)) AS freight_cost_per_unit,
  TRY_CAST(requested_delivery_date AS DATE)      AS requested_delivery_date,
  status,
  -- Derived columns use the scaled unit_price so downstream layers stay clean.
  TRY_CAST(quantity_units AS INT) * (TRY_CAST(unit_price AS DECIMAL(10,4)) / 1000)
    AS gross_revenue,
  TRY_CAST(quantity_units AS INT) * TRY_CAST(freight_cost_per_unit AS DECIMAL(10,4))
    AS total_freight_cost
FROM STREAM(<catalog>.<schema>.raw_customer_orders);

```

gold_customer_summary

Type: MATERIALIZED VIEW | **Sources:** silver_customers, silver_orders

Joins the two silver tables and aggregates to one row per customer — a classic customer 360 rollup. A MATERIALIZED VIEW is the right choice here: aggregations benefit from SDP's incremental refresh, which processes only the upstream rows that changed rather than re-running the full join and GROUP BY from scratch. Silver tables are referenced by short name; SDP resolves them and uses those references to determine that silver must run first.

What to notice

GROUP BY ALL eliminates the need to list every non-aggregated column by hand. The LEFT JOIN (not INNER JOIN) means customers with zero orders still appear in the output with NULL or zero metrics (COUNT returns 0; SUM/AVG/MIN/MAX return NULL) — important for churn analysis.

```
-- silver_customers and silver_orders referenced by short name.
-- SDP builds the DAG automatically from these references.
CREATE OR REFRESH MATERIALIZED VIEW gold_customer_summary
COMMENT "One row per active customer with lifetime order metrics."
AS
SELECT
  c.customer_id, c.customer_name, c.tier, c.region, c.contract_type,
  COUNT(DISTINCT o.order_id)      AS order_count,
  SUM(o.quantity_units)          AS total_units_sold,
  ROUND(SUM(o.gross_revenue), 2) AS lifetime_revenue,
  ROUND(SUM(o.total_freight_cost), 2) AS lifetime_freight_cost,
  ROUND(AVG(o.gross_revenue), 2) AS avg_order_revenue,
  MIN(o.order_date)              AS first_order_date,
  MAX(o.order_date)              AS last_order_date
FROM silver_customers AS c
-- LEFT JOIN: customers with zero orders still appear.
LEFT JOIN silver_orders AS o ON c.customer_id = o.customer_id
GROUP BY ALL;
```

gold_region_summary

Type: MATERIALIZED VIEW | **Source:** gold_customer_summary

Rolls the customer summary up one more level to region KPIs. Crucially, it reads from gold_customer_summary rather than re-joining the silver tables — this keeps region numbers consistent with customer numbers and makes the pipeline cheaper to compute.

What to notice

Gold layers can stack: this is a gold-on-gold pattern. `NULLIF(COUNT(*), 0)` is a defensive idiom — a `GROUP BY` can't actually produce an empty group, but the guard makes the pattern safe to reuse in other gold queries where the denominator could be zero. The `WHERE region IS NOT NULL` filter keeps the output clean for BI tools.

```
-- Builds from gold_customer_summary - gold layers can stack.
CREATE OR REFRESH MATERIALIZED VIEW gold_region_summary
COMMENT "Region-level KPIs derived from gold_customer_summary."
AS
SELECT
  region,
  COUNT(*)                AS active_customers,
  SUM(order_count)        AS total_orders,
  ROUND(SUM(lifetime_revenue), 2) AS total_revenue,
  ROUND(SUM(lifetime_freight_cost), 2) AS total_freight,
  -- NULLIF guards against divide-by-zero on an empty region.
  ROUND(SUM(lifetime_revenue) / NULLIF(COUNT(*), 0), 2) AS revenue_per_customer
FROM gold_customer_summary
WHERE region IS NOT NULL
GROUP BY region;
```